

Code-Layout, Readability and Reusability

Date	02 October 2026
Team ID	6ab7f31f8c7c66e70672939b
Project Name	EduGenie: Google Gemini Powered Learning Assistant
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	4-space indent in all Python files; consistent spacing in HTML/JS; no tabs.
2	Proper File Structure	Files and folders are logically organized	Yes	One module per feature, <i>templates/</i> and <i>static/</i> folders, <i>requirements.txt</i> .
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	Descriptive names (<i>correct_answer</i> , <i>key_points</i>); a few short ones (<i>d</i> , <i>q</i>) in helpers.
4	Function / Method Names	Functions are descriptively named	Yes	e.g. <i>clean_json_block</i> , <i>handle</i> , <i>register</i> , <i>generate</i> .
5	Code Comments	Inline and block comments explain logic	Partial	Docstrings and inline comments in <i>main.py</i> and <i>explanation_module.py</i> ; other modules have few.
6	Modular Design	Code is split into reusable functions/modules	Yes	Each feature is its own module; <i>main.py</i> holds the shared request pipeline.
7	No Redundant Code	Duplicate or unused code is removed	Partial	<i>as_str()</i> and <i>as_list()</i> are copied into 4 modules; no unused imports found.
8	Error Handling	Exceptions and errors are handled gracefully	Yes	Input validation, missing API key, timeouts, retry with model fallback, malformed AI output.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
1	handle() request pipeline (main.py)	Python / FastAPI	All 5 endpoints (/qa, /explain, /quiz, /summarize, /learn/recommendations) through register()	High
2	Feature module interface INSTRUCTION, SCHEMA, REQUIRED, clean()	Python	qna, quiz_module, summary_module, learning_path; a new feature is one new file plus one route entry	High
3	clean_json_block()	Python	Parses every Gemini response inside handle(); usable for any LLM JSON output	High
4	LearnRequest model	Python / Pydantic	Shared input validation (text length, learner level) for every endpoint	High
5	as_str() / as_list()	Python	Copied into qna, quiz_module, summary_module and learning_path instead of being shared	Medium
6	card() / copyBtn() components	JavaScript	Build every result panel and copy button in the web UI (index.html)	Medium

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	4	Clear folder layout, one module per feature, shared pipeline in <i>main.py</i> .
Readability	4	Consistent style and descriptive names; a few very long lines in <i>main.py</i> .
Reusability	3	Strong shared pipeline, but helper functions are duplicated across four modules.
Documentation / Comments	3	Docstrings in some files; few comments elsewhere and no README in the code folder.
Overall Score	3.5	Average of the four ratings above.